ESCOLA SENAI “A. JACOB LAFER”

TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

CAIO YURI

EDUARDO ZANETTI

ISABELLA RADAEL

PAOLLA VERONEZ

RAFAEL BRECCI

RAFAELA CRISTINA

PROJETO BANK 24H

SANTO ANDRÉ

2026

CAIO YURI

EDUARDO ZANETTI

ISABELLA RADAEL

PAOLLA VERONEZ

RAFAEL BRECCI

RAFAELA CRISTINA

PROJETO BANK 24H

Trabalho apresentada ao curso Técnico em Desenvolvimento de sistemas da escola SENAI A. Jacob Lafer, como requisito parcial à obtenção do título de Técnico em Desenvolvimento de sistemas. Área de concentração: Projetos

Orientadores: Prof. Raul Lopes. Prof. Paulo Camargo.

SANTO ANDRÉ
2025

AGRADECIMENTOS

A escola SENAI, pelo apoio financeiro com a manutenção da bolsa de auxílio.

Aos Professores. Raul Lopes e Paulo Camargo, pela excelente orientação.

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema bancário simulado denominado *Bank 24h*, com foco na aplicação de conceitos fundamentais de desenvolvimento web utilizando HTML, CSS e JavaScript. O projeto tem como objetivo principal implementar uma interface interativa que permita a realização de operações bancárias básicas, como saque, depósito, transferência, alteração de senha, consulta de extrato e gerador de pdf.

A proposta surge a partir da necessidade de garantir maior robustez na entrada de dados, evitando inconsistências e erros comuns causados pela ausência de validação. Para isso, são utilizadas técnicas de validação por meio de Expressões Regulares (REGEX), assegurando que apenas valores numéricos válidos sejam processados pelo sistema. Além disso, o projeto contempla o tratamento de exceções, como entradas nulas ou inválidas, e a aplicação de limites operacionais, como restrições de saldo.

O sistema também incorpora funcionalidades importantes, como autenticação de usuário baseada em login e senha previamente cadastrados, persistência de dados por meio do armazenamento do histórico de transações e atualização dinâmica da interface, proporcionando uma experiência mais próxima de sistemas reais. Outro recurso relevante é a geração de extrato com formatação monetária adequada ao padrão brasileiro.

Dessa forma, o projeto contribui para o desenvolvimento de habilidades práticas em lógica de programação, manipulação do DOM, validação de dados e organização de código, além de reforçar conceitos essenciais para a construção de aplicações seguras e eficientes.

Palavras-chave: Sistema bancário. JavaScript. Validação de dados. Expressões regulares. Interface web.

ABSTRACT

This paper presents the development of a simulated banking system called *Bank 24h*, focusing on the application of fundamental web development concepts using HTML, CSS, and JavaScript. The main objective of the project is to implement an interactive interface that allows users to perform basic banking operations, such as withdrawals, deposits, transfers, password changes, and account statement inquiries.

The proposal arises from the need to ensure greater robustness in data input, avoiding inconsistencies and common errors caused by the lack of validation. To address this issue, validation techniques using Regular Expressions (REGEX) are applied, ensuring that only valid numeric values are processed by the system. In addition, the project includes exception handling for null or invalid inputs, as well as the implementation of operational limits, such as balance restrictions.

The system also incorporates important features, such as user authentication based on previously registered login credentials, data persistence through transaction history storage, and dynamic interface updates, providing an experience closer to real-world systems. Another relevant feature is the generation of account statements with proper monetary formatting according to Brazilian standards.

Therefore, the project contributes to the development of practical skills in programming logic, DOM manipulation, data validation, and code organization, reinforcing essential concepts for building secure and efficient applications.

Keywords: Banking system. JavaScript. Data validation. Regular expressions. Web interface.

SUMÁRIO

Sumário

1 INTRODUÇÃO.....7
2 TABELA 5W2H.....8
3 METODOLOGIA ÁGIL 9
4 FIGMA 10
4.1 Paleta de cores.....10
4.1.1 Tipografia.....11
5 ESCOPO.....12
5.1 Planejamento.....13
5.1.1.1 Requisitos Funcionais.....14
5.1.1.1.1 Requisitos não funcionais 15
6 BACKLOG.....16
6.1 Sprint 1 backlog 18
6.1.1 Sprint 2 backlog 19
6.1.1.1 Sprint 3 backlog 19
Fonte: Autoria Própria 20
7 CRONOGRAMA 21
7.1 Kanban.....21
8.1 Protótipo Computador.....23
8.1.1 Protótipo Celular.....25
9 TESTE DE APLICAÇÃO 28
9.1 Imagens atualizadas do site.....29
CONCLUSÃO.....31
REFERÊNCIAS 32

1 INTRODUÇÃO

Com o avanço das tecnologias digitais, os sistemas bancários tornaram-se cada vez mais presentes no cotidiano, exigindo soluções eficientes, seguras e intuitivas para a realização de operações financeiras. Nesse contexto, o desenvolvimento de aplicações web que simulem funcionalidades bancárias contribui significativamente para a compreensão prática de conceitos fundamentais da programação e da interação entre usuário e sistema.

O presente trabalho tem como proposta o desenvolvimento de um sistema bancário simulado, denominado *Bank 24h*, utilizando as tecnologias HTML, CSS e JavaScript. A aplicação foi projetada com o objetivo de permitir ao usuário realizar operações básicas, como saque, depósito, transferência, alteração de senha e consulta de extrato, por meio de uma interface simples e funcional.

Um dos principais desafios abordados neste projeto está relacionado à validação de dados inseridos pelo usuário, uma vez que entradas incorretas podem comprometer o funcionamento do sistema. Para solucionar essa questão, foram utilizadas técnicas de validação com Expressões Regulares (REGEX), garantindo maior controle e confiabilidade no processamento das informações. Além disso, foram implementados mecanismos de tratamento de erros, como verificação de valores nulos e controle de saldo, visando evitar inconsistências.

Outro aspecto relevante do sistema é a simulação de autenticação de usuários, baseada em login e senha previamente definidos, bem como o registro do histórico de transações, permitindo o acompanhamento das operações realizadas. A atualização dinâmica da interface e a organização do código também foram consideradas, visando aproximar o projeto de aplicações reais.

Dessa forma, o desenvolvimento deste sistema proporciona a aplicação prática de conhecimentos teóricos, contribuindo para o aprimoramento das habilidades em lógica de programação, manipulação do DOM e construção de interfaces interativas, além de reforçar a importância da validação de dados e da segurança em aplicações web.

2 TABELA 5W2H

A tabela 5W2H é uma ferramenta de gestão amplamente utilizada para estruturar planos de ação de forma clara e objetiva. Originada de perguntas em inglês (What, Why, Where, When, Who, How e How much), ela organiza tarefas respondendo o quê, por quê, onde, quando, quem, como e quanto custará cada atividade.

Tem como finalidade planejar projetos, resolver problemas e monitorar responsabilidades, integrando-se a metodologias como PDCA e Lean Manufacturing. Facilita a visualização de prazos e custos em formato de tabela simples, promovendo eficiência na execução

Figura 1 – Tabela 5W2H

5W2H Projeto Sistema Bancário 24hrs						
O quê? (What?)	Por quê? (Why?)	Onde? (Where?)	Quando? (When?)	Quem? (Who?)	Como? (How?)	Quanto? (How much?)
Desenvolver a Identidade Visual e Sistema de Login com 3 Banner	Para garantir que cada instituição financeira tenha sua identidade preservada, com personalização segura e personalizada, evitando erros inferiores ao sistema.	Interfaces web (HTML/CSS/JS) voz inicial do terminal Bank 34h	Primeira e segunda quinz da projeto (09/04/2026 - 10/04/2026)	Equipe de Front-End e Designer UI/UX	Criar objeto JavaScript com dados dos 3 banco (Itaú, Santander, Nubank, Bradesco, Caixa, BB), cada um com cores, emojis e nome personalizado. Ao selecionar o banco, alterar o background via JS e linkar arquivo específico de cores e conteúdo específicos.	R$ 2.800.00 (R$ de desenvolvimento front-end + R$ 350/h)
Implementar Operações Financeiras com Validação De White e REGEX	Para blindar o sistema contra ataques de injeção (ataques, injeções, injeções angulares) garantindo que dados financeiros e permitindo que sistema só receba de dados.	Módulo de registro de operação, Saques, R$ de Saque, Depósito e Transferência	Terceira quinz da projeto (10/04/2026)	Equipe de Desenvolvimento Back-End / Lógica de Negócio	Criar lógica de validação em cada operação. Criar regras: valor, saldo com REGEX / ^d+(.d{2})?$/ - aceitar só números com 2 casas decimais e rejeitar sinais de saída. Bloquear saque acima do saldo disponível. Formatar valor com toLocaleString('pt-BR', {style:'currency', currency:'BRL'})	R$ 4.700.00 (3h de lógica + R$ 350/h)
Criar o Histórico de Transações e Extrato de Extrato	Para salvar automaticamente e transparência em tempo real, permitindo consulta rápida e compromissos com detalhes e saldo atualizado, realizados operação Bankend C da projeto.	Módulo de histórico da aplicação, array global e componente de extrato na tela	Terceira quinz da projeto (10/04/2026)	Equipe de Desenvolvimento Back-End	Definir array global 'historico[]'. A cada transação bem-sucedida, criar array parcial para registrar tipo de operação, valor, saldo resultante e data/hora. Criar função 'updateHistorico()' que faz o push e atualiza a tela no DOM com template strings.	R$ 1.750.00 (3h de desenvolvimento + R$ 350/h)
Implementar Geração de PDF do Extrato	Para que o usuário possa salvar e compartilhar o transporte em um formato mais adequado, atendendo ao requisito de valor de dados em Banco pessoal do projeto.	Módulo de transações armazenado na memória principal do Bank 34h	Terceira quinz da projeto (10/04/2026)	Equipe de Desenvolvimento Back-End	Integrar bibliotecas JavaScript (ex: jsPDF ou html2pdf.js) ao CDN. Criar função 'gerarPDF()' que converte o conteúdo do extrato implementado no DOM, opção Baixar/ao do usuário BB), e dispara o download do arquivo. Adicionar cores e cabeçalhos gerados automaticamente.	R$ 2.100.00 (4h de integração e testes + R$ 350/h) = R$ 0.00 (biblioteca open source)
Implementar Troca de Senha e Menu Completo de Operações	Para garantir segurança máxima da conta e completar todos os requisitos funcionais do sistema: Ver Saldo, Saque, Depósito, Transferência, Troca de Senha, Extrato, Geração de PDF e Sair.	Menu principal de operações, conexão de memória da memória e impressora	Terceira quinz da projeto (10/04/2026)	Equipe de Desenvolvimento Back-End	Criar função 'menuPrincipal()' com validação De White: solicitar senha atual, confirmar nova senha com REGEX de segurança (mínimo 8 dígitos). Atualizar o objeto de banco em memória. Implementar navegação de menu com switch/case ou if/else aninhado, com opção 'Sair' encerrando o servidor e limpando variáveis globais.	R$ 1.400.00 (4h dev + R$ 350/h) + R$ 700.00 (2h QA + R$ 350/h) = R$ 2.100.00

Fonte: Autoria própria

3 METODOLOGIA ÁGIL

Para o desenvolvimento do sistema *Bank 24h*, foi adotada uma abordagem baseada em metodologias ágeis, com foco na organização incremental das atividades, colaboração entre os integrantes e adaptação contínua ao longo do processo de desenvolvimento. Essa abordagem permitiu maior flexibilidade na execução do projeto, facilitando a identificação e correção de problemas de forma rápida e eficiente.

O trabalho foi estruturado em ciclos curtos de desenvolvimento, semelhantes a *sprints*, nos quais cada etapa contemplou a definição, implementação e validação de funcionalidades específicas do sistema. Inicialmente, foi realizado o levantamento dos requisitos, seguido pela definição das principais funcionalidades, como autenticação de usuário, operações bancárias e validação de dados.

Durante o desenvolvimento, as tarefas foram distribuídas entre os integrantes da equipe, considerando as competências individuais, o que contribuiu para maior produtividade e organização. Além disso, houve a utilização de ferramentas de apoio para criação de protótipos e planejamento visual, auxiliando na padronização do design e na melhor compreensão da interface do sistema.

A cada ciclo, foram realizadas revisões das funcionalidades implementadas, permitindo ajustes contínuos e melhorias no código e na interface. Esse processo iterativo foi essencial para garantir que o sistema atendesse aos requisitos propostos, além de possibilitar a evolução gradual do projeto.

Dessa forma, a utilização da metodologia ágil contribuiu significativamente para a eficiência no desenvolvimento do sistema, promovendo maior organização, adaptabilidade e qualidade no produto final, além de proporcionar uma experiência prática alinhada às demandas do mercado de tecnologia.

4 FIGMA

Pela plataforma Figma, ferramenta especializada em design de interfaces (UI), elaboramos o mapa de cores, logotipo e tipografia, visando estabelecer uma identidade visual consistente e adequada às demandas do mercado profissional.

Figura 2 – imagem do Figma

Fonte: Autoria própria

4.1 Paleta de cores

A paleta segue uma hierarquia com 6 cores principais (Primária a Senária), cada uma com três variações, um tom mais escuro (-1), o tom base e um mais claro (+1) , o que dá flexibilidade para estados de hover, fundo e destaque sem precisar improvisar cores fora do sistema.

A combinação de uma primária verde vibrante (#41FB60) com a tipografia Montserrat — que é geométrica e com personalidade forte — cria uma identidade moderna e energética. As cores mais saturadas (quaternária roxa, senária vermelha) funcionam bem em títulos grandes como H1 e H2, enquanto os tons +1 mais claros são ideais para textos auxiliares e labels menores.

Sucess e Danger formam um par semântico claro, separado das cores expressivas, o que é bom para manter consistência em feedbacks de interface.

As cores primária e secundária (verde e amarelo) têm luminosidade alta — ao usar texto sobre elas, é importante sempre recorrer aos tons escuros da própria família, como mostrado nos chips de contraste ao final da seção.

4.1.1 Tipografia

A definição tipográfica do projeto Vem de Hexa fundamenta-se na segmentação de funções visuais para otimizar a comunicação e a experiência do usuário. Como fonte primária destinada a títulos e elementos de grande impacto, adota-se a família Aclonica na hierarquia "Titulo" com tamanhos H3 20px, H2 24px e H1 40px, caracterizada por sua estrutura robusta que comunica a energia, a vibração e a informalidade intrínsecas ao universo das torcidas de futebol, estabelecendo uma assinatura visual forte e memorável para a marca. A escala tipográfica para títulos segue uma progressão que varia de small (14px) até large (40px), garantindo que a hierarquia de informações seja clara e imediata em qualquer ponto de contato com o cliente.

Para o corpo do texto e elementos de suporte informativo, utiliza-se também da família Aclonica na hierarquia "Fonte Media" nos tamanhos H2 24px e H3 20px, escolhida por sua neutralidade e alto índice de legibilidade em dispositivos digitais, características essenciais para descrições de produtos, termos de contrato e navegação em menus. A aplicação da "Fonte Media" ocorre predominantemente no tamanho medium 24px para parágrafos e small 14px para textos auxiliares, proporcionando um contraste harmônico com a densidade da "Titulo". Essa dualidade tipográfica permite que a marca transite entre o entusiasmo festivo dos títulos e a clareza técnica necessária para as transações de aluguel, assegurando uma interface profissional e equilibrada

5 ESCOPO

O presente projeto tem como escopo o desenvolvimento de um sistema bancário simulado, denominado *Bank 24h*, com a finalidade de aplicar conceitos de desenvolvimento web e lógica de programação por meio das tecnologias HTML, CSS e JavaScript. O sistema foi projetado para funcionar em ambiente web, com interface interativa e foco na experiência do usuário.

O escopo contempla a implementação de funcionalidades essenciais de um sistema bancário, incluindo autenticação de usuário por meio de login e senha previamente cadastrados, além da disponibilização de operações como saque, depósito, transferência, alteração de senha e consulta de extrato. Também está incluída a validação dos dados inseridos pelo usuário, utilizando Expressões Regulares (REGEX), com o objetivo de garantir a integridade das informações e evitar erros no processamento.

Adicionalmente, o projeto abrange o tratamento de exceções, como entradas inválidas ou nulas, bem como a aplicação de regras de negócio, como verificação de saldo disponível antes da realização de operações financeiras. O sistema também realiza o registro do histórico de transações, permitindo ao usuário acompanhar as movimentações realizadas durante a utilização da aplicação.

No que se refere à interface, o escopo inclui a construção de um layout responsivo e organizado, com atualização dinâmica dos dados exibidos, proporcionando maior interatividade. Também está prevista a formatação adequada de valores monetários, seguindo o padrão da moeda brasileira (Real).

Por outro lado, não fazem parte do escopo deste projeto a integração com bancos de dados reais, sistemas bancários oficiais ou qualquer tipo de transação financeira real. O sistema possui caráter exclusivamente educacional e simulado, não contemplando mecanismos avançados de segurança, como criptografia de dados ou autenticação em múltiplos fatores.

Dessa forma, o escopo do projeto delimita claramente suas funcionalidades e restrições, assegurando foco no aprendizado e na aplicação prática dos conceitos propostos.

5.1 Planejamento

O planejamento do projeto *Bank 24h* foi elaborado com o objetivo de organizar as etapas de desenvolvimento, definir responsabilidades e estabelecer uma sequência lógica para a implementação das funcionalidades propostas. A organização prévia das atividades foi essencial para garantir maior eficiência na execução e melhor controle do progresso do projeto.

Inicialmente, foi realizada a definição dos requisitos do sistema, identificando as principais funcionalidades a serem desenvolvidas, como autenticação de usuário, operações bancárias (saque, depósito e transferência), validação de dados e geração de extrato. Essa etapa permitiu a construção de uma base sólida para o desenvolvimento.

Em seguida, foi desenvolvido o protótipo da interface, com o objetivo de visualizar a estrutura do sistema e definir elementos como layout, cores, tipografia e organização dos componentes. Essa fase contribuiu para alinhar o design com a proposta do projeto, além de facilitar a implementação posterior.

A etapa de desenvolvimento foi dividida em partes, priorizando inicialmente a estruturação do HTML, seguida pela estilização com CSS e, por fim, a implementação das funcionalidades utilizando JavaScript. Durante esse processo, foram aplicadas técnicas de validação de dados e manipulação do DOM, garantindo o funcionamento correto das operações.

Paralelamente ao desenvolvimento, houve a distribuição de tarefas entre os integrantes da equipe, considerando as habilidades individuais, o que contribuiu para maior produtividade e organização. Também foram realizadas revisões periódicas, permitindo identificar erros, realizar ajustes e aprimorar tanto o código quanto a interface do sistema.

Por fim, foram realizados testes para verificar o funcionamento das funcionalidades implementadas, incluindo validação de entradas, operações bancárias e atualização das informações na interface. Essa etapa foi fundamental para assegurar a qualidade e a confiabilidade do sistema.

Dessa forma, o planejamento estruturado possibilitou o desenvolvimento organizado do projeto, contribuindo para o cumprimento dos objetivos propostos e para a entrega de um sistema funcional e consistente.

5.1.1 Recursos

Para o desenvolvimento do sistema *Bank 24h*, foram utilizados diversos recursos tecnológicos e ferramentas digitais que contribuíram para a organização, execução e documentação do projeto. Esses recursos foram essenciais para garantir a eficiência no desenvolvimento, a colaboração entre os integrantes da equipe e a qualidade do produto final.

No que se refere às ferramentas de prototipagem e design, foi utilizado o Figma, possibilitando a criação de interfaces visuais e a definição do layout do sistema antes da implementação. Para o gerenciamento das atividades e organização das tarefas, foi empregada a plataforma ClickUp, permitindo o acompanhamento do progresso do projeto e a distribuição das responsabilidades entre os membros da equipe.

Durante o desenvolvimento do sistema, foi utilizado o ambiente de programação Visual Studio Code (VS Code), no qual foram implementadas as linguagens HTML, CSS e JavaScript, responsáveis pela estrutura, estilização e funcionalidade da aplicação, respectivamente. Para o controle de versão e armazenamento do código-fonte, foi utilizada a plataforma GitHub, garantindo maior segurança e organização das alterações realizadas ao longo do projeto.

No que diz respeito à comunicação e colaboração entre os integrantes, foram utilizadas ferramentas como o Google Chat, para troca de informações em tempo real, e o Google Docs, para a elaboração e compartilhamento de documentos. Além disso, o Microsoft Excel foi empregado para organização de dados e planejamento, enquanto o Microsoft Word foi utilizado na formatação da documentação conforme as normas acadêmicas.

Por fim, a plataforma Canva foi utilizada para a criação de apresentações visuais e materiais gráficos, contribuindo para a exposição clara e atrativa do projeto.

Dessa forma, a utilização integrada desses recursos possibilitou maior eficiência no desenvolvimento, organização das atividades e qualidade na apresentação final do sistema.

5.1.1.1 Requisitos Funcionais

Os requisitos do sistema *Bank 24h* foram definidos com o objetivo de estabelecer claramente as funcionalidades e as condições necessárias para o correto

funcionamento da aplicação. Esses requisitos estão divididos em requisitos funcionais e não funcionais.

Os requisitos funcionais descrevem as funcionalidades que o sistema deve executar. Para este projeto, foram definidos os seguintes:

Permitir o cadastro e autenticação de usuários por meio de login e senha;

Validar os dados inseridos pelo usuário utilizando Expressões Regulares (REGEX);

Permitir a realização de operações bancárias, como:

Saque de valores, respeitando o saldo disponível; Depósito de valores; Transferência entre contas; Possibilitar a alteração de senha pelo usuário autenticado; Exibir o saldo atualizado após cada operação realizada; Gerar e exibir o extrato com o histórico de transações; Atualizar dinamicamente a interface conforme as ações do usuário; Exibir mensagens de erro em caso de dados inválidos ou operações não permitidas.

5.1.1.1.1 Requisitos não funcionais

Os requisitos não funcionais referem-se às características de qualidade e restrições do sistema. Foram definidos os seguintes:

O sistema deve possuir interface simples, intuitiva e de fácil utilização; O tempo de resposta das operações deve ser imediato, sem atrasos perceptíveis ao usuário; O sistema deve funcionar em navegadores web modernos; O código deve ser organizado, legível e estruturado, facilitando manutenção e entendimento; Os dados devem ser validados antes de qualquer processamento, garantindo a integridade das informações; O sistema deve apresentar formatação adequada de valores monetários no padrão brasileiro (Real); A aplicação deve operar de forma local, sem necessidade de conexão com banco de dados externo; O sistema deve garantir simulação segura das operações, mesmo sem implementar mecanismos avançados de segurança.

Dessa forma, a definição dos requisitos contribui para o desenvolvimento estruturado do sistema, garantindo que as funcionalidades e características atendam aos objetivos propostos.

6 BACKLOG

O Product Backlog é um artefato central da metodologia ágil Scrum, que consiste em uma lista ordenada e dinâmica de todos os requisitos, funcionalidades e melhorias necessárias para o desenvolvimento de um produto. Ele representa o trabalho total a ser realizado pela equipe e serve como única fonte de verdade sobre o que precisa ser construído. Cada item do backlog, denominado User Story ou simplesmente item, descreve uma funcionalidade sob a perspectiva do valor que entrega ao usuário ou ao negócio. O Product Backlog é continuamente refinado ao longo do projeto, podendo ter itens adicionados, removidos ou reordenados conforme as necessidades evoluem.

No projeto Bank 24h, o Product Backlog foi estruturado em dez funcionalidades principais, detalhadas a seguir.

Figura 3- Imagem 1 do product backlog

Código	Funcionalidade	Etapas	Sub Etapas	Descrição da Etapa
1	Identidade Visual e Seleção de Banco	Desenvolver protótipo no Figma	Criar telas de seleção de banco → Definir paleta de cores por banco → Aplicar identidade visual	Criar interface inicial com os 6 bancos
		Validar protótipo com o time	Apresentar protótipo → Coletar feedback → Ajustar layout e cores	Garantir usabilidade e identidade de cada banco
		Definir estrutura HTML/CSS	Criar HTML base → Estilizar cards de seleção → Configurar objeto JS com dados dos bancos	Estruturar base visual do terminal
		Implementar troca dinâmica de cor	Capturar evento de seleção → Aplicar cor hexadecimal via JS → Testar em todos os bancos	Alterar background conforme banco selecionado
2	Login e Autenticação	Desenvolver protótipo da tela de login	Criar tela no Figma → Definir campos usuário/senha → Aplicar identidade do banco selecionado	Criar interface de acesso seguro
		Validar protótipo com o time	Revisar com equipe → Ajustar campos e mensagens de erro → Aprovar layout	Garantir clareza no fluxo de login
		Implementar autenticação JS	Criar formulário de login → Comparar credenciais com objeto JS → Exibir mensagem de erro se inválido	Validar usuário e senha do banco
		Implementar controle de sessão	Armazenar banco ativo em variável global → Bloquear acesso ao menu sem login → Criar função de logout	Manter sessão segura durante uso

Fonte: Autoria Própria

Figura 4- Imagem 2 do product backlog

3	Menu de Operações Bancárias	Validar protótipo com o time	Revisar opções do menu → Ajustar hierarquia visual → Aprovar layout	Garantir navegação intuitiva
		Implementar estrutura HTML do menu	Criar div principal → Adicionar botões: Ver Saldo, Saque, Depósito, Transferência, Troca de Senha, Extrato, PDF, Sair → Aplicar CSS	Montar todos os botões do terminal
		Integrar botões às funções JS	Vincular onClick de cada botão → Testar chamada de funções → Validar fluxo completo do menu	Conectar interface às funcionalidades
4	Saque com Do While e REGEX	Desenvolver protótipo da operação	Criar fluxo de saque no Figma → Definir mensagens de erro → Definir confirmação	Criar interface da operação de saque
		Validar protótipo com o time	Testar fluxo → Ajustar mensagens de erro → Aprovar experiência do usuário	Garantir clareza no fluxo de saque
		Implementar lógica do saque	Criar função sacar() → Usar do{...}while → Capturar valor com prompt() → Validar com REGEX /^\d+(\.\d+)?$/ → Verificar saldo suficiente e valor > 0	Blindar entrada e processar saque com segurança
		Registrar transação e atualizar saldo	Subtrair valor do saldo global → Registrar no array histórico com Date() → Atualizar saldo na tela com toLocaleString BRL	Persistir operação e exibir saldo atualizado

Fonte: Autoria Própria

Figura 5- Imagem 3 do product backlog

5	Depósito com Do While e REGEX	Desenvolver protótipo da operação	Criar fluxo de depósito no Figma → Definir mensagens de validação → Definir confirmação	Criar interface da operação de depósito
		Validar protótipo com o time	Testar fluxo → Ajustar mensagens → Aprovar experiência	Garantir clareza no fluxo de depósito
		Implementar lógica do depósito	Criar função depositar() → Usar do{...}while → Capturar valor com prompt() → Validar com REGEX → Verificar se valor > 0	Blindar entrada e processar depósito
		Registrar transação e atualizar saldo	Somar valor ao saldo global → Registrar no array histórico com Date() → Atualizar saldo na tela com formatação BRL	Persistir operação e exibir saldo atualizado
6	Transferência com Do While e REGEX	Desenvolver protótipo da operação	Criar fluxo de transferência no Figma → Definir campos de valor e destino → Definir confirmação	Criar interface da operação de transferência
		Validar protótipo com o time	Testar fluxo → Ajustar campos → Aprovar experiência do usuário	Garantir clareza no fluxo de transferência
		Implementar lógica da transferência	Criar função transferir() → Usar do{...}while → Capturar valor com prompt() → Validar com REGEX → Verificar saldo suficiente e valor > 0	Blindar entrada e processar transferência
		Registrar transação e atualizar saldo	Subtrair valor do saldo → Registrar destino e valor no array histórico → Atualizar saldo com formatação BRL	Persistir operação e exibir saldo atualizado

Fonte: Autoria Própria

Figura 6- Imagem 4 do product backlog

7	Troca de Senha	Desenvolver protótipo da operação	Criar fluxo de troca de senha no Figma → Definir campos de senha atual e nova → Definir confirmação	Criar interface de segurança de senha
		Validar protótipo com o time	Testar fluxo → Ajustar mensagens de erro → Aprovar experiência	Garantir clareza e segurança no fluxo
		Implementar lógica de troca de senha	Criar função trocarSenha() → Usar do{...}while → Solicitar senha atual → Validar com REGEX (mín. 6 dígitos) → Confirmar nova senha → Atualizar objeto do banco em memória	Permitir alteração segura da senha
		Testar e validar fluxo completo	Testar senha incorreta → Testar senha fraca → Testar troca bem-sucedida → Verificar login com nova senha	Garantir segurança da operação
8	Histórico de Transações	Desenvolver protótipo da tela de histórico	Criar layout de extrato no Figma → Definir campos: tipo, valor, saldo, data/hora → Aplicar identidade visual	Criar interface de consulta de transações
		Validar protótipo com o time	Revisar layout → Ajustar ordenação e campos → Aprovar interface	Garantir legibilidade do extrato
		Implementar array de histórico	Declarar array global historico[] → Em cada operação bem-sucedida usar array.push() com tipo, valor, saldo e new Date() formatado	Registrar todas as movimentações
		Implementar exibição do extrato	Criar função exibirExtrato() → Iterar array → Renderizar itens no DOM com template strings → Formatar valores em BRL	Exibir histórico de transações na tela

Fonte: Autoria Própria

Figura 7- Imagem 5 do product backlog

9	Geração de PDF do Extrato	Desenvolver protótipo da saída PDF	Definir layout do comprovante no Figma → Definir campos: banco, usuário, transações, saldo final, data	Criar modelo do comprovante em PDF
		Validar protótipo com o time	Revisar leiaute do PDF → Ajustar campos → Aprovar modelo	Garantir clareza do comprovante
		Integrar biblioteca de geração de PDF	Importar jsPDF ou html2pdf.js via CDN → Criar função gerarPDF() → Capturar conteúdo do extrato no DOM → Configurar nome do arquivo com data/hora automática	Habilitar exportação do extrato
		Testar e validar geração do arquivo	Testar download com histórico vazio → Testar com múltiplas transações → Verificar formatação BRL no PDF → Validar nome do arquivo gerado	Garantir funcionamento da exportação
10	Protótipo Figma e Documentação	Definir estrutura e fluxo das telas	Mapear fluxo completo do sistema → Organizar telas: seleção, login, menu, operações, extrato → Definir navegação	Organizar fluxo completo do Bank 24h
		Criar protótipo completo no Figma	Desenvolver todas as telas → Aplicar identidade visual de cada banco → Criar componentes reutilizáveis	Desenvolver design de alta fidelidade
		Validar protótipo com equipe	Apresentar todas as telas → Coletar feedback → Ajustar fluxo e identidade visual	Alinhar protótipo com requisitos do projeto
		Finalizar documentação do projeto	Revisar 5W2H → Atualizar Kanban → Consolidar cronograma → Preparar relatório de distribuição de atividades → Subir artefatos no GitHub	Entregar documentação completa do projeto

Fonte: Autoria Própria

6.1 Sprint 1 backlog

A Sprint 1 concentrou os itens relacionados à base visual e estrutural do sistema. As funcionalidades trabalhadas foram a identidade visual e seleção de banco, o login e autenticação, o menu de operações bancárias e a prototipagem completa no Figma. Nessa etapa foram criados os protótipos de todas as telas, definidas as paletas de cores por banco, estruturado o HTML/CSS base e implementada a autenticação com controle de sessão, estabelecendo a fundação sobre a qual as demais sprints seriam desenvolvidas.

Figura 8- Imagem 1 do backlog sprint 1

Sprint 1 — Fundação Visual, Acesso e Prototipação				
Código	**Funcionalidade**	**Etapas**	**Sub Etapas**	**Descrição da Etapa**
1	Identidade Visual e Seleção de Banco	Desenvolver protótipo no Figma	Criar telas de seleção de banco → Definir paleta de cores por banco → Aplicar identidade visual	Criar interface inicial com os 6 bancos
		Validar protótipo com o time	Apresentar protótipo → Coletar feedback → Ajustar layout e cores	Garantir usabilidade e identidade de cada banco
		Definir estrutura HTML/CSS	Criar HTML base → Estilizar cards de seleção → Configurar objeto JS com dados dos bancos	Estruturar base visual do terminal
		Implementar troca dinâmica de cor	Capturar evento de seleção → Aplicar cor hexadecimal via JS → Testar em todos os bancos	Alterar background conforme banco selecionado
2	Login e Autenticação	Desenvolver protótipo da tela de login	Criar tela no Figma → Definir campos usuário/senha → Aplicar identidade do banco selecionado	Criar interface de acesso seguro
		Validar protótipo com o time	Revisar com equipe → Ajustar campos e mensagens de erro → Aprovar layout	Garantir clareza no fluxo de login
		Implementar autenticação JS	Criar formulário de login → Comparar credenciais com objeto JS → Exibir mensagem de erro se inválido	Validar usuário e senha do banco
		Implementar controle de sessão	Armazenar banco ativo em variável global → Bloquear acesso ao menu sem login → Criar função de logout	Manter sessão segura durante uso
3	Menu de Operações Bancárias	Desenvolver protótipo do menu	Criar tela no Figma → Definir 8 botões de operações → Aplicar identidade visual do banco	Criar interface central de navegação
		Validar protótipo com o time	Revisar opções do menu → Ajustar hierarquia visual → Aprovar layout	Garantir navegação intuitiva
		Implementar estrutura HTML do menu	Criar div principal → Adicionar botões: Ver Saldo, Saque, Depósito, Transferência, Troca de Senha, Extrato, PDF, Sair → Aplicar CSS	Montar todos os botões do terminal
		Integrar botões às funções JS	Vincular onClick de cada botão → Testar chamada de funções → Validar fluxo completo do menu	Conectar interface às funcionalidades

Fonte: Autoria Própria

Figura 9 – Imagem 2 do backlog sprint 1

10	Protótipo Figma e Documentação	Definir estrutura e fluxo das telas	Mapear fluxo completo do sistema → Organizar telas: seleção, login, menu, operações, extrato → Definir navegação	Organizar fluxo completo do Bank 24h
		Criar protótipo completo no Figma	Desenvolver todas as telas → Aplicar identidade visual de cada banco → Criar componentes reutilizáveis	Desenvolver design de alta fidelidade
		Validar protótipo com equipe	Apresentar todas as telas → Coletar feedback → Ajustar fluxo e identidade visual	Alinhar protótipo com requisitos do projeto
		Finalizar documentação do projeto	Revisar 5W2H → Atualizar Kanban → Consolidar cronograma → Preparar relatório de distribuição de atividades → Subir artefatos no GitHub	Entregar documentação completa do projeto

6.1.1 Sprint 2 backlog

A Sprint 2 focou na implementação das operações financeiras principais do terminal. As funcionalidades desenvolvidas foram saque, depósito e transferência, todas seguindo a mesma lógica de validação obrigatória por meio da estrutura do{...}while combinada com Expressões Regulares. Cada operação garante que apenas valores numéricos válidos sejam processados, verificando também as regras de saldo antes de confirmar qualquer transação e registrando cada movimentação no histórico do sistema.

Figura 10 – Imagem do backlog sprint 2

Código	Funcionalidade	Etapas	Sub Etapas	Descrição da Etapa
4	Saque com Do While e REGEX	Desenvolver protótipo da operação	Criar fluxo de saque no Figma → Definir mensagens de erro → Definir confirmação	Criar interface da operação de saque
		Validar protótipo com o time	Testar fluxo → Ajustar mensagens de erro → Aprovar experiência do usuário	Garantir clareza no fluxo de saque
		Implementar lógica do saque	Criar função sacar() → Usar do{...}while → Capturar valor com prompt() → Validar com REGEX /^\d+(\.\d+)?$/ → Verificar saldo suficiente e valor > 0	Blindar entrada e processar saque com segurança
		Registrar transação e atualizar saldo	Subtrair valor do saldo global → Registrar no array histórico com Date() → Atualizar saldo na tela com toLocaleString BRL	Persistir operação e exibir saldo atualizado
5	Depósito com Do While e REGEX	Desenvolver protótipo da operação	Criar fluxo de depósito no Figma → Definir mensagens de validação → Definir confirmação	Criar interface da operação de depósito
		Validar protótipo com o time	Testar fluxo → Ajustar mensagens → Aprovar experiência	Garantir clareza no fluxo de depósito
		Implementar lógica do depósito	Criar função depositar() → Usar do{...}while → Capturar valor com prompt() → Validar com REGEX → Verificar se valor > 0	Blindar entrada e processar depósito
		Registrar transação e atualizar saldo	Somar valor ao saldo global → Registrar no array histórico com Date() → Atualizar saldo na tela com formatação BRL	Persistir operação e exibir saldo atualizado
6	Transferência com Do While e REGEX	Desenvolver protótipo da operação	Criar fluxo de transferência no Figma → Definir campos de valor e destino → Definir confirmação	Criar interface da operação de transferência
		Validar protótipo com o time	Testar fluxo → Ajustar campos → Aprovar experiência do usuário	Garantir clareza no fluxo de transferência
		Implementar lógica da transferência	Criar função transferir() → Usar do{...}while → Capturar valor com prompt() → Validar com REGEX → Verificar saldo suficiente e valor > 0	Blindar entrada e processar transferência
		Registrar transação e atualizar saldo	Subtrair valor do saldo → Registrar destino e valor no array histórico → Atualizar saldo com formatação BRL	Persistir operação e exibir saldo atualizado

Fonte: Autoria Própria

6.1.1.1 Sprint 3 backlog

A Sprint 3 reuniu as funcionalidades finais do projeto, voltadas à segurança e à entrega de informações ao usuário. Foram implementadas a troca de senha com validação por REGEX, o histórico de transações com registro em array global e exibição do extrato na tela, e a geração de PDF do comprovante de movimentações. Essa sprint encerrou o ciclo de desenvolvimento do Bank 24h, entregando todas as funcionalidades previstas no Product Backlog geral.

Figura 11 – Imagem do backlog sprint 3

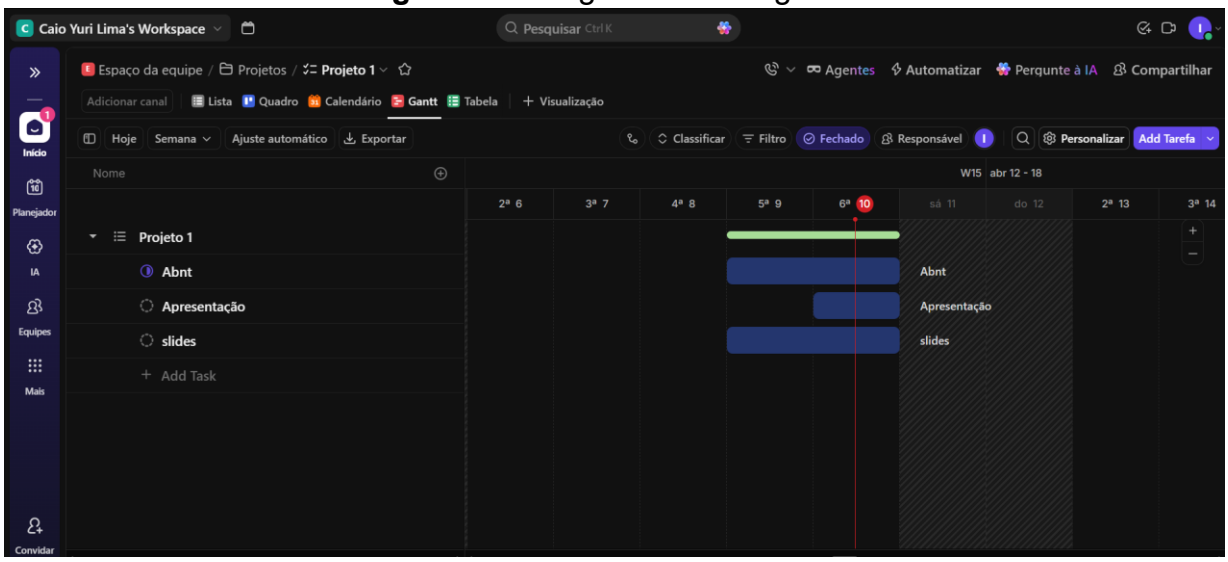
Sprint 3 — Segurança, Histórico, PDF e Encerramento				
Código	**Funcionalidade**	**Etapas**	**Sub Etapas**	**Descrição da Etapa**
7	Troca de Senha	Desenvolver protótipo da operação	Criar fluxo de troca de senha no Figma → Definir campos de senha atual e nova → Definir confirmação	Criar interface de segurança de senha
		Validar protótipo com o time	Testar fluxo → Ajustar mensagens de erro → Aprovar experiência	Garantir clareza e segurança no fluxo
		Implementar lógica de troca de senha	Criar função trocarSenha() → Usar do{...}while → Solicitar senha atual → Validar com REGEX (mín. 6 dígitos) → Confirmar nova senha → Atualizar objeto do banco em memória	Permitir alteração segura da senha
		Testar e validar fluxo completo	Testar senha incorreta → Testar senha fraca → Testar troca bem-sucedida → Verificar login com nova senha	Garantir segurança da operação
8	Histórico de Transações	Desenvolver protótipo da tela de histórico	Criar layout de extrato no Figma → Definir campos: tipo, valor, saldo, data/hora → Aplicar identidade visual	Criar interface de consulta de transações
		Validar protótipo com o time	Revisar layout → Ajustar ordenação e campos → Aprovar interface	Garantir legibilidade do extrato
		Implementar array de histórico	Declarar array global historico[] → Em cada operação bem-sucedida usar array.push() com tipo, valor, saldo e new Date() formatado	Registrar todas as movimentações
		Implementar exibição do extrato	Criar função exibirExtrato() → Iterar array → Renderizar itens no DOM com template strings → Formatar valores em BRL	Exibir histórico de transações na tela
9	Geração de PDF do Extrato	Desenvolver protótipo da saída PDF	Definir layout do comprovante no Figma → Definir campos: banco, usuário, transações, saldo final, data	Criar modelo do comprovante em PDF
		Validar protótipo com o time	Revisar leiaute do PDF → Ajustar campos → Aprovar modelo	Garantir clareza do comprovante
		Integrar biblioteca de geração de PDF	Importar jsPDF ou html2pdf.js via CDN → Criar função gerarPDF() → Capturar conteúdo do extrato no DOM → Configurar nome do arquivo com data/hora automática	Habilitar exportação do extrato
		Testar e validar geração do arquivo	Testar download com histórico vazio → Testar com múltiplas transações → Verificar formatação BRL no PDF → Validar nome do arquivo gerado	Garantir funcionamento da exportação

Fonte: Autoria Própria

7 CRONOGRAMA

Para aprimorar a gestão temporal do projeto, desenvolvemos um cronograma detalhado que possibilita a alocação eficiente e o acompanhamento preciso das jornadas de trabalho de cada desenvolvedor, implementado por meio da plataforma ClickUp.

Figura 12- Imagem do cronograma



Fonte: Print retirado do Site: https://app.clickup.com/

7.1 Kanban

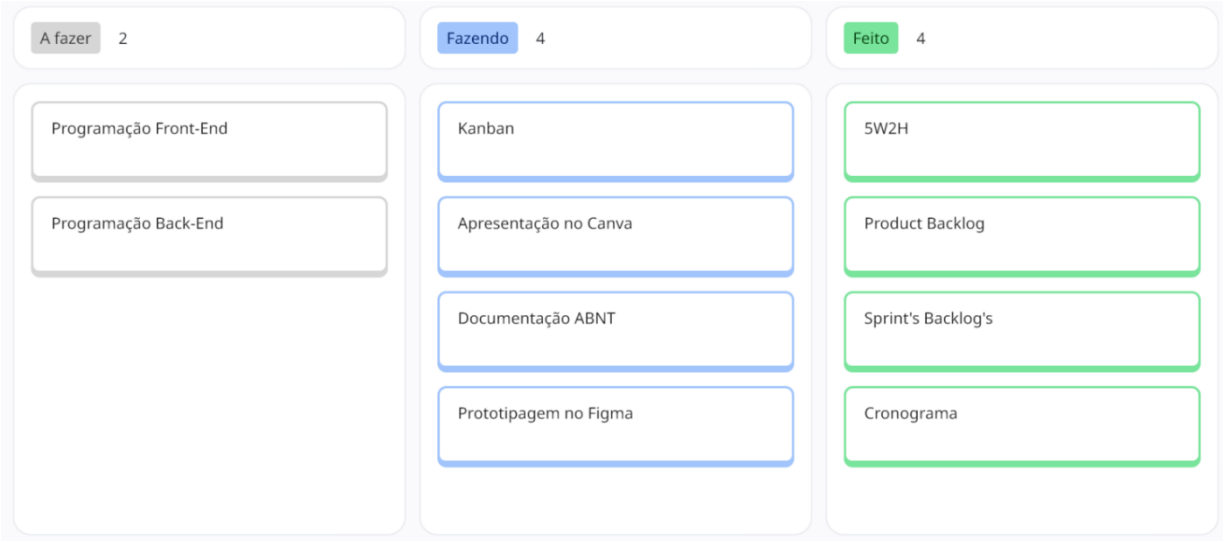
Para o gerenciamento das atividades do projeto, foi utilizado o método Kanban, organizado em três colunas: *A fazer*, *fazendo* e *Feito*. Essa abordagem permitiu o acompanhamento visual do progresso das tarefas ao longo do desenvolvimento.

Na coluna *Feito*, foram concluídas as etapas de planejamento e organização do projeto, contemplando a elaboração do 5W2H, a construção do Product Backlog, a definição dos Sprint's Backlogs e a montagem do Cronograma. Na coluna *Fazendo*, encontram-se em andamento as atividades de desenvolvimento do Kanban, criação

da Apresentação no Canva, elaboração da Documentação ABNT e Prototipagem no Figma. Por fim, na coluna *A fazer*, aguardam execução as etapas de Programação Front-End e Programação Back-End.

Essa divisão evidencia que o projeto encontra-se em fase intermediária, com a base documental consolidada e o desenvolvimento técnico previsto para as próximas etapas

Figura 13 - Imagem do Kanban



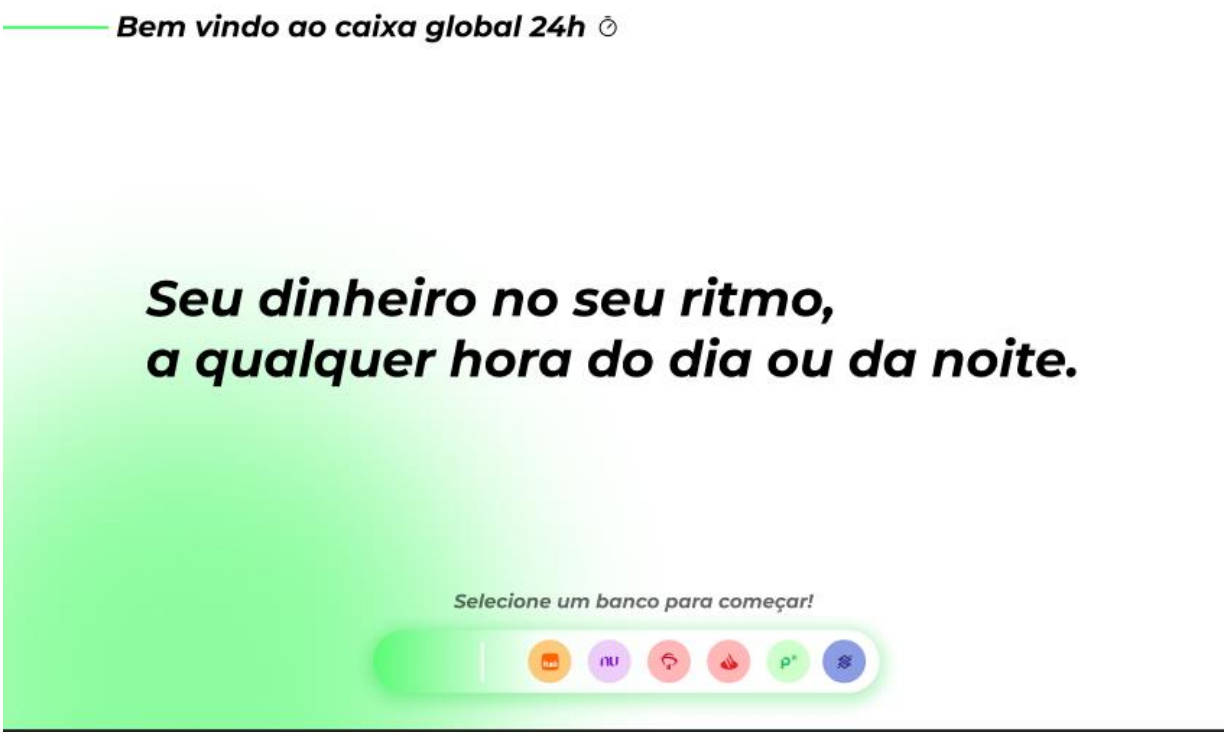
Fonte: Print retirado do Site: https://miro.com/pt/

8 PROTÓTIPO FIGMA

Para aprimorar a compreensão visual do projeto, elaboramos um protótipo interativo na plataforma Figma. Este protótipo possibilita a navegação fluida entre todas as telas, incorpora efeitos hover interativos nos botões e apresenta um design funcional e inovador, retratando de maneira precisa e detalhada o comportamento completo do sistema web.

8.1 Protótipo Computador

Figura 14 – Página inicial



Fonte: Print retirado do protótipo Figma

Figura 15 - Sistema do Banco escolhido

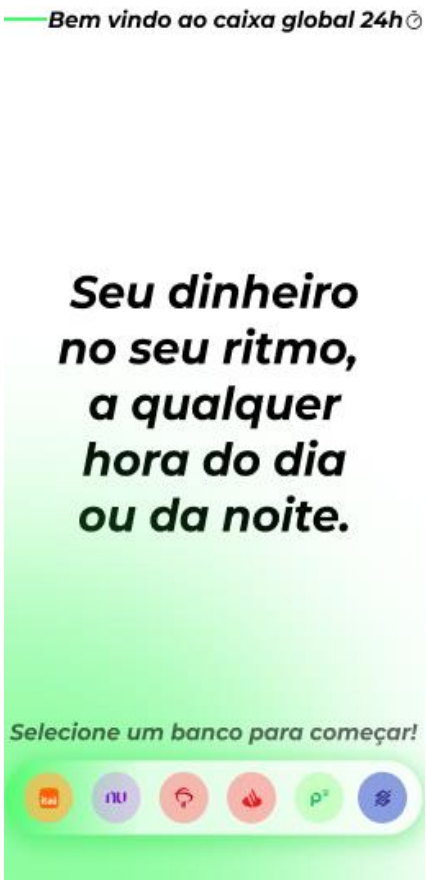
Fonte: Print retirado do protótipo Figma

Figura 16 – Página de extrato

Fonte: Print retirado do protótipo Figma

8.1.1 Protótipo Celular

Figura 17 – Página inicial mobile

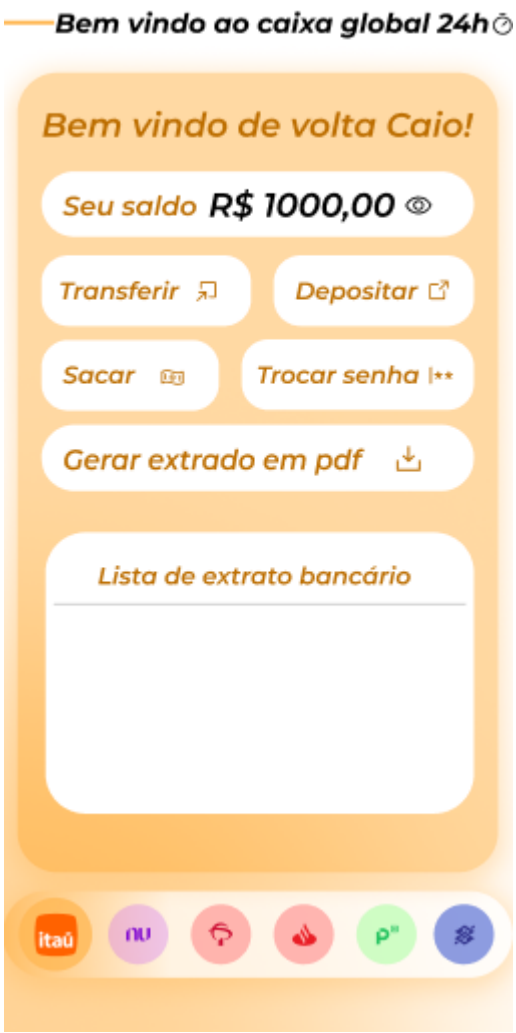


Fonte: Print Retirado do Protótipo Figma

Figura 18 – Página de Cadastro

Fonte: Print Retirado do Figma

Figura 19 – Página do extrato mobile



Fonte: Print Retirado do Figma

9 TESTE DE APLICAÇÃO

A etapa de testes do sistema Bank 24h foi realizada com o objetivo de verificar o correto funcionamento das funcionalidades implementadas, bem como garantir a confiabilidade e a integridade dos dados processados. Os testes foram baseados em atividades práticas previamente desenvolvidas, com destaque para a aplicação de validações utilizando Expressões Regulares (REGEX).

Inicialmente, foram realizados testes de validação de entrada de dados, com foco na verificação de campos numéricos e textuais. Esses testes tiveram como base atividades desenvolvidas anteriormente, nas quais foram aplicadas expressões regulares para garantir que apenas valores válidos fossem aceitos pelo sistema. Dessa forma, foi possível evitar erros comuns, como inserção de caracteres inválidos em campos destinados a valores monetários.

Além disso, foram testadas as funcionalidades principais do sistema, incluindo operações de saque, depósito e transferência. Durante esses testes, foram analisadas situações como saldo insuficiente, valores negativos e entradas nulas, assegurando que o sistema apresentasse respostas adequadas para cada cenário, como mensagens de erro ou bloqueio da operação.

A funcionalidade de autenticação também foi validada, verificando o comportamento do sistema diante de credenciais corretas e incorretas. Esse processo garantiu que apenas usuários autorizados pudessem acessar as funcionalidades disponíveis.

Outro aspecto testado foi a geração do extrato, no qual foi verificado o registro correto das transações realizadas, bem como a atualização dinâmica das informações na interface. Também foram realizados testes relacionados à alteração de senha, garantindo que o sistema aplicasse corretamente as mudanças solicitadas pelo usuário.

Os testes foram conduzidos de forma manual, com a simulação de diferentes cenários de uso, permitindo identificar possíveis falhas e realizar ajustes no sistema. A utilização de atividades anteriores, como a implementação de validações com REGEX, contribuiu significativamente para a eficiência dessa etapa.

Dessa forma, os testes realizados foram fundamentais para assegurar a qualidade da aplicação, garantindo que o sistema operasse de maneira consistente, segura e conforme os requisitos estabelecidos.

9.1 Imagens atualizadas do site

O sistema *Bank 24h* consiste em uma aplicação web interativa desenvolvida com o objetivo de simular operações bancárias básicas em um ambiente digital. O site foi construído utilizando as tecnologias HTML, CSS e JavaScript, sendo estruturado de forma a proporcionar uma navegação simples, intuitiva e funcional ao usuário.

A interface do sistema apresenta um layout inspirado em terminais bancários, com elementos visuais que remetem a um ambiente tecnológico, como cores escuras, tipografia monoespaçada e organização centralizada dos componentes. Essa escolha estética contribui para uma experiência imersiva, aproximando o usuário de um sistema real.

Ao acessar o site, o usuário é direcionado para uma área de autenticação, onde deve inserir login e senha previamente cadastrados. Após a validação das credenciais, o sistema libera o acesso às funcionalidades disponíveis, garantindo uma camada básica de controle de acesso.

Dentro do ambiente principal, o usuário pode realizar diversas operações bancárias, como saque, depósito e transferência, por meio de comandos simples. Cada ação realizada resulta na atualização imediata das informações exibidas na tela, incluindo o saldo disponível e o histórico de transações. O sistema também permite a alteração de senha, reforçando a interatividade e o controle do usuário sobre sua conta.

Outro recurso importante do site é a exibição do extrato, que registra todas as operações realizadas durante a sessão. Os valores são apresentados com formatação adequada ao padrão monetário brasileiro, facilitando a leitura e compreensão das informações.

O site conta ainda com mecanismos de validação de dados, impedindo a inserção de valores inválidos e garantindo maior segurança na execução das operações. Em casos de erro, o sistema exibe mensagens informativas ao usuário, orientando sobre a correção necessária.

Dessa forma, o *Bank 24h* se caracteriza como uma aplicação web funcional e didática, que simula de maneira eficiente as principais operações de um sistema bancário, ao mesmo tempo em que proporciona uma experiência prática no desenvolvimento de interfaces interativas e no tratamento de dados em tempo real.

Figura 18 – Tela Inicial

Fonte: Autoria Própria

Figura 19 – Tela Bem-vindo

Fonte: Autoria Própria

Figura 19-

Fonte: Autoria Própria

Figura 20 – Cadastro de Produtos

Fonte: Autoria Própria

Figura 21 – Listagem de produtos

Fonte: Autoria Própria

CONCLUSÃO

O desenvolvimento do sistema *Bank 24h* possibilitou a aplicação prática de conceitos fundamentais da área de desenvolvimento web, integrando conhecimentos de HTML, CSS e JavaScript na construção de uma aplicação funcional e interativa. Ao longo do projeto, foi possível compreender de forma mais aprofundada a importância da organização do código, da validação de dados e da experiência do usuário na criação de sistemas digitais.

Um dos principais destaques do trabalho foi a implementação de mecanismos de validação utilizando Expressões Regulares (REGEX), que contribuíram significativamente para a prevenção de erros e para a garantia da integridade das informações inseridas no sistema. Além disso, o tratamento de exceções e a aplicação de regras de negócio, como o controle de saldo, reforçaram a importância da lógica de programação no desenvolvimento de aplicações seguras e eficientes.

A adoção de uma abordagem baseada em metodologias ágeis também se mostrou relevante, permitindo maior organização das atividades, divisão de tarefas entre os integrantes da equipe e realização de melhorias contínuas ao longo do processo. O uso de ferramentas digitais contribuiu para a colaboração, planejamento e documentação do projeto, tornando o desenvolvimento mais estruturado.

Os testes realizados, baseados em atividades práticas anteriores, foram essenciais para validar o funcionamento do sistema e identificar possíveis ajustes, garantindo maior confiabilidade na execução das funcionalidades propostas. Dessa forma, o projeto atendeu aos objetivos estabelecidos, entregando uma aplicação capaz de simular operações bancárias básicas de maneira eficiente.

Por fim, destaca-se que, apesar de seu caráter educacional, o sistema desenvolvido se aproxima de aplicações reais, proporcionando uma experiência significativa de aprendizado. O projeto contribuiu para o aprimoramento das habilidades técnicas e reforçou a importância de boas práticas no desenvolvimento de software, servindo como base para projetos futuros mais complexos.

REFERÊNCIAS

TAVARES, Breno Gontijo; SILVA, Carlos Eduardo Sanches da. Análise bibliométrica de artigos científicos sobre a utilização de metodologias ágeis na gestão de projetos. In: SIMPÓSIO DE ENGENHARIA DE PRODUÇÃO, 19., 2012, Bauru. Anais [...]. Bauru: UNESP, 2012.

A Plataforma de Colaboração Visual para Todas as Equipes | Miro. Disponível em: <https://miro.com/pt/>.

CANVA. *Canva: design gráfico online*. Disponível em: https://www.canva.com/. Acesso em: 09 abr. 2026.

CLICKUP. *ClickUp: ferramenta de gerenciamento de tarefas*. Disponível em: https://clickup.com/. Acesso em: 09 abr. 2026.

FIGMA. *Figma: ferramenta de design de interfaces*. Disponível em: https://www.figma.com/. Acesso em: 09 abr. 2026.

GITHUB. *GitHub: plataforma de hospedagem de código-fonte*. Disponível em: https://github.com/. Acesso em: 09 abr. 2026.

GOOGLE. *Google Docs: editor de documentos online*. Disponível em: https://docs.google.com/. Acesso em: 09 abr. 2026.

GOOGLE. *Google Chat: ferramenta de comunicação*. Disponível em: https://chat.google.com/. Acesso em: 09 abr. 2026.

MICROSOFT. *Microsoft Excel*. Disponível em: https://www.microsoft.com/. Acesso em: 09 abr. 2026.

MICROSOFT. *Microsoft Word*. Disponível em: https://www.microsoft.com/. Acesso em: 09 abr. 2026.

MOZILLA. *MDN Web Docs: HTML, CSS e JavaScript*. Disponível em: https://developer.mozilla.org/pt-BR/. Acesso em: 09 abr. 2026.

W3SCHOOLS. *W3Schools Online Web Tutorials*. Disponível em: https://www.w3schools.com/. Acesso em: 09 abr. 2026.